

Plano objetivo — Heurísticas de experiência do usuário no LangNet

Destinatário: Claude Code / equipe de evolução do LangNet

Escopo: BioByte como validação inicial; implementação genérica para qualquer domínio

Status: planejamento; não implementar sem aprovação

1. Decisão

O LangNet já possui arquétipos de tela, coerência entre caso de uso, UI Spec e Modelo de Dados, protótipo React navegável, apontamento de componentes e refinamento por conversa.

Não criar uma etapa isolada chamada “Design Thinking”. Incorporar um **Contrato de Experiência da Tela** nas etapas existentes:

```
Especificação → define pessoa, contexto, objetivo e decisão
UI Spec       → define hierarquia, componentes e estados
Protótipo     → permite observar, apontar e refinar
Testes/Portões → verificam se a experiência é executável e compreensível
```

O modelo pode propor. O programa deve registrar e conferir.

2. Lacunas que a mudança deve resolver

As heurísticas atuais reconhecem mapa, gráfico, upload, tabela, formulário, dashboard, galeria, kanban e timeline. Ainda faltam contratos explícitos para:

- contexto real de uso;
- objetivo do usuário;
- decisão apoiada pela tela;
- prioridade e hierarquia da informação;
- estados inicial, carregando, vazio, sucesso e erro;
- falha de serviço externo e recuperação;
- acessibilidade básica;
- teste de compreensão e conclusão da tarefa;
- justificativa da escolha do arquétipo.

Não resolver isso com a frase genérica “aplique Design Thinking”. A instrução precisa gerar dados estruturados e verificáveis.

3. Onde acrescentar no pipeline

3.1 Requisitos — contexto da pessoa

Camada: documento de requisitos e análise inicial.

Para cada ator relevante, registrar quando a fonte fornecer:

- papel e responsabilidade;
- situação em que usa o sistema;
- objetivo prático;
- frequência da atividade;
- pressão, risco ou custo do erro;
- informação que precisa primeiro;
- resultado que considera sucesso;
- restrições de dispositivo, conectividade ou acessibilidade.

Não inventar personas. Se a informação não existir, registrar “não informado” ou uma pergunta.

Saída: seção Contexto de Uso e Perfis, rastreável aos requisitos.

3.2 Especificação — objetivo, decisão e jornada

Arquivo principal: `backend/app/templates/specification_prompt.py`.

Adicionar a cada caso de uso:

```
Contexto de uso
Objetivo do usuário
Decisão apoiada
Informação necessária antes da ação
Critério de sucesso percebido pelo usuário
Riscos e consequências de erro
Jornada resumida
```

A jornada deve ser curta e concreta, por exemplo:

```
perceber risco → localizar caso → conferir evidências → analisar resultado
→ decidir conduta → registrar decisão → acompanhar consequência
```

Para cada tela do UC, declarar também:

- ação primária;
- ações secundárias;
- informação crítica e informação de apoio;
- estados de espera, vazio, sucesso e falha;
- mensagem e ação de recuperação.

Essas informações complementam o fluxo principal; não o substituem.

3.3 UI Spec — Contrato de Experiência da Tela

Arquivos principais:

- `backend/prompts/generate_ui_spec.py`;
- `backend/agents/langnetui.py`;
- `backend/agents/langnetcoherence.py`;
- `src/pages/UISpecPage.tsx`.

Adicionar ao JSON de cada tela:

```
{
  "experience": {
    "user_goal": "O que o usuário precisa conseguir fazer",
    "user_context": "Em que situação a tela é usada",
    "primary_decision": "Qual decisão a tela apoia",
    "success_criteria": ["Critério observável de sucesso"],
    "priority": "critical | important | supporting",
    "information_hierarchy": ["resultado", "evidências", "detalhes", "ações"],
    "design_rationale": "Por que o arquétipo foi escolhido",
    "states": {
      "initial": "Estado antes da ação",
      "loading": "O que aparece durante processamento",
      "empty": "O que aparece sem dados",
      "success": "Como o resultado aparece",
      "error": "Como o erro aparece",
      "external_failure": "Como falha de MCP/API aparece",
      "recovery_action": "Como tentar novamente ou corrigir"
    },
    "accessibility": {
      "keyboard_navigation": true,
      "visible_labels": true,
      "status_not_only_by_color": true
    }
  }
}
```

O gerador não deve preencher com frases genéricas. Campos obrigatórios ausentes devem ser marcados como pendência ou reprovação, nunca silenciosamente inventados.

3.4 Escolha do arquétipo

Arquivo principal: `backend/prompts/generate_ui_spec.py`

Função relacionada: `infer_screen_capabilities()`

Manter os arquétipos atuais, mas decidir nesta ordem:

1. intenção e decisão do caso de uso;
2. interação descrita no fluxo e no wireframe;
3. tipo dos dados no Modelo de Dados;
4. palavras-chave, somente como último recurso.

Não deixar uma palavra isolada decidir. “Relatório”, por exemplo, pode exigir tabela, gráfico, documento exportável ou combinação desses elementos.

Registrar `design_rationale` com uma justificativa verificável:

```
Dashboard porque o UC compara indicadores agregados por período.
Upload/prévia porque o ator importa um arquivo e precisa conferir seu conteúdo
antes de processar.
Detail porque o ator avalia evidências de um caso antes de executar a ação.
```

Permitir composição de arquétipos quando necessário:

```
detail + metric-card + chart + timeline
```

3.5 Portão determinístico de experiência

Arquivo principal: `backend/agents/langnetcoherence.py`.

Adicionar verificações determinísticas para:

- objetivo do usuário presente;
- decisão presente quando o UC exige decisão;
- ação primária identificável;
- hierarquia da informação não vazia;
- campos do fluxo presentes na tela;
- resultado principal não tratado como entrada;
- estado vazio em listas, tabelas e dashboards;
- estado de carregamento em tarefas agênticas;
- estado de erro em tarefas com exceção;
- falha externa em telas com MCP/API;
- recuperação quando o erro for recuperável;
- labels visíveis nos campos;
- status não dependente apenas de cor;
- justificativa compatível com o arquétipo.

Os problemas devem ser nomeados para o usuário, por exemplo:

“Monitor de Integração” usa MCP, mas não declara estado de falha externa.
“Dashboard de Vigilância” possui métricas, mas não declara hierarquia da informação.
“Importar Microbiologia” possui upload, mas não declara prévia nem recuperação de erro.

O sistema deve propor a correção e permitir aprovação, como já faz com binds do Modelo de Dados.

3.6 Aplicativo gerado — estados reais

Arquivo principal: `backend/agents/langnetagents.py`.

O emissor React deve consumir `experience.states` e emitir, quando aplicável:

- indicador de carregamento;
- estado vazio honesto;
- resultado principal destacado;
- mensagem de sucesso;
- mensagem de erro funcional;
- falha de integração externa sem mock;
- ação de recuperação;
- confirmação para ações destrutivas;
- separação visual entre entrada, evidência e resultado.

Dados fictícios continuam permitidos no protótipo, mas não podem esconder ausência de dados no aplicativo implantado.

O contrato de tela deve conferir também os estados obrigatórios, além dos componentes declarados.

3.7 Protótipo — observação e refino

Arquivos principais: `backend/agents/langnetprototype.py` e `src/pages/UISpecPage.tsx`.

Preservar o fluxo:

```
apontar componente → pedir mudança → remontar → observar resultado
```

Adicionar sugestões objetivas ao painel de refinamento:

- A ação principal está clara?
- O resultado mais importante aparece primeiro?
- O que aparece enquanto o sistema processa?
- O que aparece quando não há dados?
- O usuário sabe como se recuperar do erro?
- Algum campo deveria ser somente leitura?

As respostas devem virar refinamento registrado da tela, não comentário perdido no chat.

3.8 Casos de Teste — experiência observável

Arquivo principal: `backend/agents/langnettest.py`.

Para cada UC crítico, acrescentar pelo menos um teste que verifique:

- identificação da ação principal;
- conclusão da tarefa;
- exibição do resultado principal;
- percepção do carregamento;
- distinção entre estado vazio e erro;
- mensagem de erro acionável;
- distinção entre falha externa e falha de dados;
- possibilidade de tentar novamente;
- distinção entre entrada, evidência e resultado.

Quando houver tarefa real, executar o comportamento. Não aprovar procurando apenas uma frase no HTML.

O resultado deve indicar:

```
{
  "onde": "tela | sistema | externo",
  "espera": "conclusao | compreensao | recusa | recuperacao",
  "criterio": "Texto observável",
```

```
"resultado": "passou | falhou | nao_exercitavel"
}
```

4. Ordem de implementação

Fase A — Contrato

1. Definir o esquema `experience`.
2. Definir campos obrigatórios por tipo de tela.
3. Validar o schema.
4. Manter compatibilidade com telas antigas, marcando-as como legado/pendente.

Fase B — Especificação

1. Atualizar `specification_prompt.py`.
2. Preservar contexto, objetivo, decisão, sucesso e estados na geração em fases.
3. Conferir rastreabilidade dessas informações aos requisitos.

Fase C — UI Spec e portão

1. Atualizar `generate_ui_spec.py`.
2. Emitir `experience` e `design_rationale`.
3. Atualizar `langnetcoherence.py`.
4. Exibir pendências na UI e exigir aprovação para mudanças estruturais.

Fase D — Protótipo e aplicativo

1. Emitir os estados no React gerado.
2. Atualizar a conferência do contrato de tela.
3. Expor estados no protótipo.
4. Permitir refino por componente e por estado.

Fase E — Teste E2E

1. Criar testes de experiência para os UCs críticos.
2. Executar sucesso, vazio, carregamento, erro e falha externa.
3. Executar o encadeamento real pela Rede de Petri.
4. Registrar os resultados na validação e no roteiro da apresentação.

5. Prioridade no BioByte

Aplicar primeiro em:

1. Login com MFA;
2. Importação de microbiologia;
3. Detalhe do caso clínico;
4. Classificação NHSN;
5. Detecção de MDR;
6. Escore de Cox;

7. Recomendação de tratamento;
8. Dashboard de vigilância;
9. Exportação de relatório;
10. Monitor de integrações.

6. O que não fazer

- Não criar uma etapa independente apenas chamada Design Thinking.
- Não colocar toda a decisão em uma frase genérica do prompt.
- Não inventar personas que não estejam na fonte.
- Não transformar todas as telas em formulário ou dashboard.
- Não usar dados fictícios para esconder falhas do aplicativo real.
- Não aceitar “parece bom” como validação.
- Não alterar o Modelo de Dados automaticamente sem proposta e aprovação.
- Não sobrescrever versões aprovadas.

7. Critérios de aceite

A melhoria estará concluída quando:

- UCs críticos tiverem contexto, objetivo e decisão;
- cada tela possuir `experience`;
- o arquétipo tiver justificativa rastreável;
- o portão detectar estados ausentes;
- o aplicativo renderizar vazio, carregamento, erro e recuperação;
- falhas de MCP/API forem visíveis e honestas;
- o protótipo permitir revisar esses estados;
- cada UC crítico possuir teste de experiência executável;
- a rastreabilidade continuar íntegra;
- o BioByte passar pela execução E2E sem regressão.

8. Resultado esperado

O LangNet deverá responder não apenas:

“A tela tem os campos certos?”

Mas também:

“A pessoa certa consegue tomar a decisão certa, no contexto certo, entende o resultado e sabe o que fazer quando algo dá errado?”

Essa é a contribuição objetiva de Design Thinking ao pipeline: transformar contexto humano, decisão e recuperação em artefatos versionados e verificáveis, compatíveis com a Rede de Petri e com a geração técnica do aplicativo.